CICD pipeline and Monitoring Tools
Summary Report

Details	Student Entry
Student Name	Pratham Kadam
Roll Number(s)	DS 14 (PRN: 2324000362)
Department / Class	Final Year B.Tech CSE (Data Science) - Semester VII
Academic Year	A.Y 2026-27
GitHub Repository URL	https://github.com/prathham-k21/mldevops-cicd.git

1. Problem Statement

Develop an end-to-end Machine Learning Web Application for Heart Disease Risk Prediction (CardioPulse) and implement the DevOps lifecycle on a Windows PC. The project utilizes Scikit-Learn for training a robust clinical risk classification model and Flask for delivering both a responsive web interface and REST API endpoints. Use Git for version control, GitHub for source-code hosting, Docker for application containerization, and Jenkins for automating the build, testing, and deployment process (including automated pytest suites). In addition, Prometheus monitoring and telemetry endpoints are integrated to observe real-time latency, request rates, and prediction distribution. Configure each tool, verify its installed version, and execute the necessary Windows commands to understand how the application moves from development to deployment.

2. Tools: Environment Setup

1. Python and Flask -- Web Application Development

1. Environment Setup

- Install Python and configure the system PATH.
- Install Flask, Scikit-Learn, Pandas, Joblib, Prometheus Client, and Pytest.
- Train and verify the Heart Disease model via train.py, generating model artifact.
- Verify the installed versions of Python and pip, and run unit tests.

2. Commands to Run on Windows

- `python --version`
- `pip --version`
- `pip install -r requirements.txt`
- `python train.py`
- `python -m pytest -v`
- `python app.py`

2. Git -- Version Control

1. Environment Setup

- Install Git for Windows.
- Configure the Git username and email, and verify the installed Git version.

2. Commands to Run on Windows

- `git --version`
- `git init`
- `git status`
- `git add .`
- `git commit -m "feat: complete MLOps CI/CD & Monitoring project with report"`

3. GitHub -- Source-Code Hosting

1. Environment Setup

- Create a GitHub account and a remote repository.
- Configure Git authentication using an appropriate method, such as Git Credential Manager or SSH.

2. Commands to Run on Windows

Configured with the actual URL of the GitHub repository:

- `git remote add origin https://github.com/prathham-k21/mldevops-cicd.git`
- `git branch -M main`
- `git push -u origin main`
- `git pull origin main`

4. Docker -- Application Containerization

1. Environment Setup

- Install Docker Desktop and configure WSL 2 if required.
- Verify the Docker Engine status and installed Docker and Docker Compose versions.
- Construct Dockerfile specifying base image, dependencies, exposed ports, and healthcheck probe.

2. Commands to Run on Windows

- `docker --version`
- `docker compose version`
- `docker info`
- `docker build -t cardiopulse-ml-app .`
- `docker run -d -p 5000:5000 --name cardiopulse-service cardiopulse-ml-app`
- `docker ps`

5. Jenkins -- Continuous Integration and Continuous Deployment

1. Environment Setup

- Install a Java JDK version compatible with the selected Jenkins release.
- Install and configure Jenkins, and verify the Java and Jenkins versions.
- Configure the required Git and Docker integrations for the pipeline using Jenkinsfile.

2. Commands to Run on Windows

For Jenkins installed directly on Windows:

- `java -version`
- `git --version`
- `docker --version`
- `jenkins.exe start -- only if using a Jenkins installation that provides this service command.`

For Jenkins running as a Docker container named jenkins:

- `docker run -d -p 8080:8080 -p 50000:50000 --name jenkins jenkins/jenkins:lts`
- `docker ps`
- `docker logs jenkins`
- `docker start jenkins`
- `docker exec jenkins java -version`

6. Monitoring Tools -- Prometheus & Health Telemetry

1. Environment Setup

- Integrate prometheus-client into Flask application to collect real-time HTTP metrics.
- Configure health probe endpoint (/health) for container liveness and readiness verification.
- Expose Prometheus telemetry endpoint (/metrics) to track latency and prediction drift.

2. Commands to Run on Windows

- `curl -s http://localhost:5000/health`
- `curl -s http://localhost:5000/metrics`
- `curl -X POST http://localhost:5000/predict -H "Content-Type: application/json" -d "{\"age\":55,.`